

06. Micro: bit

Parcours 1, 2, 3 et 4



Description de l'activité

Au travers des 4 parcours, les élèves découvrent la carte micro:bit et ses spécificités techniques. Chaque parcours se complexifie et utilise de nouveaux capteurs.

Remarque : les parcours se basent sur le déroulé proposé ici.

Objectifs pédagogiques ou compétences

Objectifs généraux	Objectifs intermédiaires	Compétences
Programmation	- Séquences d'instructions - Appels de fonctions - Boucles répéter - Instructions conditionnelles	- Comprendre une instruction et la traduire en programme - Analyser une erreur et déboguer un programme - Envisager des usages concrets - (fac) Programmer de zéro un objet simulé

Matériel et outils

- Ordinateurs et connexion Internet : sites [Vittascience](#) ou [Micro:bit](#)
- 1 carte et un câble Micro:bit par élève ou binôme
- Fiche activité élève à imprimer

Tags

#objetsConnectés ; #simulateurs ; #cartesMicrobit ; #pixels ; #boucles ; #actionneurs

Déroulé de l'activité

Introduction : (~15 minutes)

- Présenter les objectifs de la séance (contenu théorique et productions attendues) (2-3 minutes)
- Introduire la carte : (~10 minutes)

Dans un premier temps, on présente la carte Micro:bit (MB). A priori, certains élèves en auront déjà utilisé, on peut leur demander d'expliquer son fonctionnement et ses fonctionnalités.

Ce qu'il faut retenir :

- 25 LEDs programmables individuellement
- 2 boutons programmables + 1 bouton tactile (le logo)
- Broches de connexion
- Capteurs de lumière et de température
- Microphone
- Capteurs de mouvements (accéléromètre et boussole)
- Mini haut-parleur
- Communication sans fil, via radio et Bluetooth
- Interface USB
- Grâce à un Shield, on peut augmenter ses possibilités : robot Maqueen, guirlande LEDs, ajout de servo-moteurs, capteur d'humidité, écrans LCD, ...
- Elle peut être programmée par blocs, mais aussi en python.

L'enseignant.e présente ensuite l'interface aux élèves.

On peut ensuite leur demander ce qu'ils ont déjà fait avec ces cartes, et de quelle manière ils l'ont programmée (blocs ? code ?). On peut également leur demander d'imaginer des projets qu'on pourrait faire avec la carte MB à partir des fonctionnalités citées plus tôt. Voici quelques exemples :

- **Thermomètre connecté** : afficher la valeur, ou même envoyer les infos via radio ou Bluetooth.
- **Capteur de lumière** : en fonction de la luminosité (et même par paliers), on peut allumer les LEDs, ou même commander une ou plusieurs guirlandes LED.
- **Capteur d'humidité** : on peut créer un système qui prévient quand la terre des plantes est trop sèche, et même créer un système d'arrosage automatique avec des servo-moteurs et des tubes.
- **Détecteur de mouvements** : on peut l'utiliser pour créer une alarme, mais aussi un jeu par exemple où le joueur doit reproduire une suite de mouvements.
- **Accéléromètre** : on peut créer un compteur de pas.
- Etc.

Parcours 1 – Découverte (~1h)

REMARQUE 1 : ON PEUT PROGRAMMER SUR 2 PLATEFORMES, [Vittascience](#) ou [Micro:bit](#). L'AVANTAGE DU SECOND EST DE NE PAS IMPOSER LA CONSOLE ET D'AVOIR UN ACCES DIRECT A LA BIBLIOTHEQUE DE COMMANDES. C'EST AVEC CELUI-CI QUE CETTE ACTIVITE A ETE CREEE.

REMARQUE 2 : SUIVANT LES FONCTIONNALITES MOBILISEES, ON PEUT PASSER SEULEMENT PAR LE SIMULATEUR POUR ALLER PLUS VITE, OU BIEN TESTER EGALEMENT SUR LA CARTE MICRO:BIT.

- **Afficher un texte et une image**

Les élèves testent avec le texte et l'image proposés, puis modifient le code à leur guise. Ensuite, ils créent leur propre image, puis utilisent les coordonnées.

1.

```
1 from microbit import *
2
3 while True:
4     display.scroll('Hello, World!')
5     display.show(Image.HEART)
6     sleep(2000)
```

2.

```
1 from microbit import *
2 display.scroll("mauriac", delay=20)
```

3.

```
1 from microbit import *
2
3 bateau = Image("05050:"
4               "05050:"
5               "05050:"
6               "99999:"
7               "09990")
8
9 display.show(bateau)
```

4.

```
1 from microbit import *
2 display.set_pixel(1, 4, 9)
```

- **Boucles**

Les élèves testent les boucles « while » et « for » puis modifient les programmes. Pour la boucle « for », on leur demande quelle modification faire pour que le pixel ne se déplace que sur les 3 premières LEDs.

5.

```
1 from microbit import *
2 while True:
3     for i in range(5):
4         display.set_pixel(i,0,9)
5         sleep(200)
6         display.clear()
```

6.

```
1 from microbit import *
2 while True:
3     display.set_pixel(2, 2, 9)
4     sleep(500)
5     display.clear()
6     sleep(500)
```

- **Exercice 1 :**

On peut ensuite demander aux élèves de faire défiler un pixel d'abord sur la première ligne puis sur la première colonne, enfin il d'afficher un losange.

Correction :

```
from microbit import *

while True:
    for i in range(5):
        display.set_pixel(i,0,9)
        sleep(200)
        display.clear()

    for i in range (5) :
        display.set_pixel(0,i,9)
        sleep(200)
        display.clear()

    display.show(Image('00300:'
                       '03630:'
                       '36963:'
                       '03630:'
                       '00300'))

    sleep(400)
    display.clear()
```

- **Exercice 2 :**

Les élèves sont libres de tester l'effet de leur choix, par exemple animer un personnage (Pacman etc.) qui arrive à gauche, traverse l'écran et disparaît à droite.

Parcours 2 – Actionneurs (~2h)

- **Boutons A et B**

On présente les programmes liés aux boutons A, B, A+B, et les deux commandes possibles : `is_pressed()` ou `was_pressed()`. On présente le programme et la commande « break », puis on laisse les élèves expérimenter.

```
1 from microbit import *
2 while True:
3     display.scroll("SNT")
4     sleep(200)
5     if button_a.was_pressed():
6         break
7 display.clear()
8 display.show(Image.SAD)
```

7.

- **Exercice 1**

Écrire un programme permettant de basculer d'une image à une autre suivant qu'on appuie sur A ou sur B.

Correction :

```
1 from microbit import *
2 while True:
3     if button_a.was_pressed():
4         display.show(Image.SAD)
5     if button_b.was_pressed():
6         display.show(Image.HAPPY)
```

La commande « `if button_a.is_pressed() and button_b.is_pressed():` » permet de cumuler les 2 boutons.

- **Exercice 2**

Niveau 1 : Écrire un programme permettant de déplacer un point vers la gauche ou vers la droite en appuyant sur A ou sur B.

Correction :

```
1 from microbit import *
2 x = 2
3 while True:
4     display.clear()
5     display.set_pixel(x, 2, 9)
6     if button_a.was_pressed():
7         x = x - 1
8     if button_b.was_pressed():
9         x = x + 1
```

Niveau 2 : Écrire un programme permettant de faire parcourir tout l'écran au pixel :

- Si on sort à droite, on se décale d'une ligne vers le bas et on revient tout à gauche.
- Si on sort à gauche, on se décale d'une ligne vers le haut et on revient tout à droite.

Correction :

```

1  from microbit import *
2  x = 2
3  y = 2
4  while True:
5      display.clear()
6      display.set_pixel(x, y, 9)
7      if button_a.was_pressed():
8          x = x - 1
9      if button_b.was_pressed():
10         x = x + 1
11     if x == 5:
12         x = 0
13         y = y + 1
14     if x == -1:
15         x = 4
16         y = y - 1
17     if y == 5:
18         x = 0
19         y = 0
20     if y == -1:
21         x = 4
22         y = 4

```

● Exercice 3

On veut créer le jeu suivant :

- Au démarrage, un pixel aléatoire est placé sur l'écran.
- Il faut ensuite se déplacer d'un point vers la gauche ou vers la droite en appuyant sur A ou sur B.
- Lorsque qu'on a rejoint le point aléatoire, un emoji HAPPY apparaît.

Correction :

```

1  from microbit import *
2  from random import randint
3
4  n = randint(0,4)
5  p = randint(0,4)
6
7  x = 2
8  y = 2
9
10 while True:
11     display.clear()
12     display.set_pixel(n, p, 9)
13     display.set_pixel(x, y, 9)
14
15     if button_a.was_pressed():
16         x = x - 1
17
18     if button_b.was_pressed():
19         x = x + 1
20
21     if x == 5:
22         x = 0
23         y = y + 1
24
25     if x == -1:
26         x = 4
27         y = y - 1
28
29     if y == 5:
30         y = 0
31
32     if y == -1:
33         y = 4
34
35     if x == n and y == p:
36         display.show(Image.HAPPY)
37         break

```

Parcours 3 – Lumières automatiques (~1h)

Grâce à la commande `display.read_light_level()`, l'écran LED devient un capteur de lumière basique, permettant de détecter la luminosité ambiante. La commande retourne un entier compris entre 0 et 255 représentant le niveau de lumière.

● Capteur de lumière

L'enseignant.e demande comment fonctionne une lumière automatique (si la luminosité ambiante dépasse un niveau donné, la lumière s'éteint), puis guide les élèves sur la manière de traduire ça en logique de programmation : si le niveau de luminosité est satisfaisant, s'éteindre, sinon, s'allumer (ou l'inverse). L'enseignant.e explique le principe de la condition « if ... else » et écrit petit à petit le programme avec la classe :

```
1  from microbit import *
2
3  soleil = Image("90909:"
4                "09990:"
5                "99999:"
6                "09990:"
7                "90909:")
8
9  lune = Image("00999:"
10             "09990:"
11             "09900:"
12             "09990:"
13             "00999:")
14
15  while True:
16      if display.read_light_level() > ... : #trouver la bonne valeur (entre 0 et 255)
17          display.show(soleil)
18      else:
19          display.show(...) #trouver la bonne variable
20      sleep(10)
```

● Eclairage progressif

Les élèves créent des paliers progressifs, (niveau 0 à 5), avec de plus en plus de LEDs allumées. Pour tester le fonctionnement, on peut regarder avec une lampe-torche ou la lampe du téléphone portable.


```

1  from microbit import *
2
3  lvl0 = Image("00000:"
4              "00000:"
5              "00000:"
6              "00000:")
7
8  lvl1 = Image("00000:"
9              "00000:"
10             "00000:"
11             "00000:"
12             "99999:")
13
14  lvl2 = Image("00000:"
15              "00000:"
16              "00000:"
17              "99999:"
18              "99999:")
19
20
21  lvl3 = Image("00000:"
22              "00000:"
23              "99999:"
24              "99999:"
25              "99999:")
26
27  lvl4 = Image("00000:"
28              "99999:"
29              "99999:"
30              "99999:"
31              "99999:")
32
33  lvl5 = Image("99999:"
34              "99999:"
35              "99999:"
36              "99999:"
37              "99999:")
38
39  while True:
40      if display.read_light_level() > 0 and display.read_light_level() < 40 :
41          display.show(lvl5)
42      else:
43          if display.read_light_level() > 40 and display.read_light_level() < 80 :
44              display.show(lvl4)
45          else:
46              if display.read_light_level() > 80 and display.read_light_level() < 120 :
47                  display.show(lvl3)
48              else:
49                  if display.read_light_level() > 160 and display.read_light_level() < 200 :
50                      display.show(lvl2)
51                  else:
52                      if display.read_light_level() > 200 and display.read_light_level() < 240 :
53                          display.show(lvl1)
54                      else :
55                          if display.read_light_level() > 240 :
56                              display.show(lvl0)

```

● Microphone :

Grâce au microphone, on peut programmer un système d'allumage/extinction des lumières en tapant des mains. L'enseignant.e peut donner l'exercice sous forme de code à compléter, ou bien reconstruire la logique pas à pas avec la classe.

```

1  from microbit import *
2
3  led_display_on = True
4
5  while True:
6      mic_value = microphone.sound_level()
7
8      if mic_value > 50:
9          led_display_on = not led_display_on
10         if led_display_on:
11             display.show(Image.HAPPY)
12         else:
13             display.clear()

```

Conclusion (~10 minutes)

- **Discussion :** (5 minutes)

Pour conclure, on peut demander aux élèves de revenir sur les points clés retenus, les points faciles / difficiles / intéressants de l'activité.

On peut également les sonder sur d'éventuels projets qu'ils aimeraient faire en programmation.

- **Les métiers en lien :** (10 minutes)

On peut évoquer les différents métiers en lien avec les objets connectés.

Exemples de métiers :

- **Développeur IoT (Internet des objets) :**
 - Programmation des logiciels embarqués dans les objets connectés.
 - Conception d'applications mobiles et de serveurs pour gérer les objets connectés.
- **Ingénieur en électronique embarquée :**
 - Conception des circuits électroniques et des composants pour les objets connectés.
- **Ingénieur en télécommunications :**
 - Mise en place des réseaux de communication (Wi-Fi, Bluetooth, 4G/5G) pour connecter les objets entre eux et à Internet.
- **Designer d'expérience utilisateur (UX) :**
 - Conception des interfaces utilisateur pour les applications mobiles et les tableaux de bord de gestion des objets connectés.
- **Ingénieur en sécurité IoT :**
 - Protection des données et sécurisation des objets connectés contre les attaques.
- **Technicien de maintenance IoT :**
 - Installation, configuration et maintenance des objets connectés sur le terrain.
- **Data Scientist IoT :**
 - Analyse des données collectées par les objets connectés pour en extraire des informations pertinentes.
- **Ingénieur en énergie pour l'IoT :**
 - Conception de solutions d'alimentation énergétique efficaces pour les objets connectés.
- **Ingénieur en automatisation :**
 - Programmation de systèmes d'automatisation et de contrôle basés sur l'IoT.
- **Consultant en IoT :**
 - Conseil aux entreprises sur la manière d'intégrer les objets connectés dans leurs opérations et leur stratégie.

EVALUATION :

Programmer une
simulation d'objet
connecté.

Les élèves choisissent un objet de leur choix à simuler par la programmation. On peut également leur demander de présenter leur objet à la classe :

- L'objet, son utilité et son fonctionnement
- Le programme (points clés)
- Les difficultés rencontrées et ce dont l'élève est fier.e

Exemples d'objets à simuler sous Scratch :

- Lumières automatiques / portes automatiques qui s'activent quand un personnage entre dans une zone donnée.
- Chauffage et climatisation automatiques qui s'activent et se désactivent en fonction de la température.
- Aspirateur autonome qui nettoie un déchet dès qu'il apparaît.
- Une alarme à incendie dans une zone qui fait venir un drone « pompier »
- Caméra qui détecte une personne âgée en détresse (chute, inconscience, ...) et appelle les secours

...

Programme ton objet multifonctions

Fiche activité élève

Avec les différentes fonctionnalités vues en classe, mais aussi en effectuant des recherches sur d'autres fonctionnalités (boussole, accéléromètre, buzzer, etc.), imagine un objet multifonctions et programme-le.

- **Quelle est l'utilité principale de ton objet ?**

- **Quelles sont ses autres fonctionnalités ? (*minimum 2*)**

- **Pour chaque fonctionnalité, liste les boutons et/ou capteurs utilisés par le programme.**

- **Qu'est-ce qui a été facile à faire, ou que tu penses avoir bien réussi ?**

.....

.....

.....

.....

.....

.....

.....

.....

- **Qu'est-ce qui t'a posé des difficultés ?**

.....

.....

.....

.....

.....

.....

.....

.....